

# Product Roadmap & Implementation Plan

Phased goals, objectives, deliverables, exit conditions and implementation order

## 1. Roadmap Strategy

The roadmap is organized around capabilities rather than a flat feature checklist. Each phase has a goal, objectives, deliverables and an exit condition. The sequence minimizes rework and avoids introducing advanced infrastructure before the core domain is stable.

### Phase 0 - Product & Architecture Foundation

Goal: Create the technical foundation used by every later feature.

#### Objectives

- Define domain boundaries and API conventions.
- Set up Go project structure, configuration and dependency management.
- Implement HTTP server, middleware, request context, error handling, request IDs and graceful shutdown.
- Set up PostgreSQL connectivity, migrations and health/readiness endpoints.
- Establish OpenAPI conventions.

#### Key deliverables

- Project structure
- Configuration
- HTTP server
- Middleware
- Request context
- Logging baseline
- Database connection
- Migrations
- Health/readiness
- API v1 skeleton
- OpenAPI skeleton

Exit condition: The service starts cleanly, connects to PostgreSQL, uses standardized middleware, shuts down gracefully and exposes a documented v1 API skeleton.

### Phase 1 - Authentication

Goal: Provide secure identity and session management.

#### Objectives

- Implement registration/login.
- Implement password hashing.
- Implement JWT access tokens.

- Implement refresh tokens and session tracking.
- Implement logout.
- Add email verification and password reset.

### **Key deliverables**

- Registration
- Login
- JWT
- Refresh tokens
- Sessions
- Logout
- Password reset
- Email verification

Exit condition: A user can register, authenticate, refresh a session and log out through documented APIs.

## **Phase 2 - Multi-Tenancy**

Goal: Establish organization-based tenant isolation.

### **Objectives**

- Implement organizations and memberships.
- Implement invitations/member management.
- Define organization roles.
- Centralize tenant resolution and authorization.
- Add cross-tenant security tests.

### **Key deliverables**

- Organizations
- Memberships
- Invitations
- Organization roles
- Tenant isolation

Exit condition: A user can operate in an organization and cannot access another organization's protected resources.

## **Phase 3 - RBAC & Project Authorization**

Goal: Implement organization and project-level permissions.

### **Objectives**

- Define the permission matrix.
- Implement Owner, Admin, Project Manager, Member and Viewer behavior.
- Add project membership.
- Enforce permissions at service/domain boundaries.

## Key deliverables

- Permission model
- Organization RBAC
- Project RBAC
- Project membership
- Authorization tests

Exit condition: Every protected organization/project action has a defined permission and automated tests.

## Phase 4 - Projects, Tasks & Collaboration

Goal: Deliver the first complete product workflow.

### Objectives

- Build project CRUD.
- Build full-featured tasks.
- Implement assignment, status, priority, labels and due dates.
- Implement comments.
- Add validation and authorization.

## Key deliverables

- Projects
- Tasks
- Statuses
- Priorities
- Assignees
- Labels
- Due dates
- Comments

Exit condition: A complete organization -> project -> task -> comment workflow works end to end.

## Phase 5 - Query Infrastructure

Goal: Make APIs useful for real frontend workloads.

### Objectives

- Implement pagination.
- Implement filtering and sorting.
- Implement PostgreSQL-backed search.
- Create reusable query conventions.

## Key deliverables

- Pagination
- Filtering

- Sorting
- Search

Exit condition: Collection endpoints expose consistent query behavior with validation and documentation.

## Phase 6 - Redis

Goal: Introduce distributed state and performance capabilities.

### Objectives

- Implement rate limiting.
- Add caching to justified read paths.
- Use Redis for session/token state.
- Introduce distributed locks only for concrete concurrency needs.

### Key deliverables

- Rate limiting
- Caching
- Session/token state
- Distributed locks

Exit condition: Redis is integrated with tests, TTL policies, failure handling and operational visibility.

## Phase 7 - Kafka & Background Processing

Goal: Move asynchronous work out of request paths.

### Objectives

- Define domain events.
- Introduce Kafka topics and consumer groups.
- Implement notification processing.
- Add retries and dead-letter strategy.
- Create worker architecture for future jobs.

### Key deliverables

- Kafka
- Domain events
- Consumers
- Notification worker
- Retries
- Dead-letter handling
- Maintenance workers

Exit condition: Asynchronous work can be produced and consumed reliably without blocking the core API.

## Phase 8 - Notifications

Goal: Deliver meaningful email events asynchronously.

## Objectives

- Implement task assignment notifications.
- Implement comment/mention notifications.
- Implement organization invitation notifications.
- Implement password reset and verification emails.
- Add due-soon reminders.

## Key deliverables

- Email templates
- Event consumers
- Delivery failure handling
- Reminder processing

Exit condition: Defined business events result in reliable asynchronous notification processing.

## Phase 9 - Observability

Goal: Make the system diagnosable in production-like conditions.

### Objectives

- Standardize structured logs.
- Add correlation/request IDs.
- Expose Prometheus metrics.
- Build Grafana dashboards.
- Add OpenTelemetry traces.
- Track dependency latency and Kafka lag.

### Key deliverables

- Logs
- Metrics
- Dashboards
- Tracing
- Correlation IDs
- Health/readiness

Exit condition: Common latency, error and dependency problems can be diagnosed from telemetry.

## Phase 10 - Testing

Goal: Build confidence through automated validation.

### Objectives

- Unit-test domain logic.
- Add PostgreSQL/Redis/Kafka integration tests.
- Add API/E2E workflows.

- Use Testcontainers.
- Add load and security test foundations.

## Key deliverables

- Unit tests
- Integration tests
- E2E/API tests
- Testcontainers
- Load tests
- Security tests

Exit condition: Core workflows and infrastructure integrations are protected by automated tests.

## Phase 11 - Docker & CI/CD

Goal: Automate reproducible development and delivery.

### Objectives

- Containerize the application.
- Create a local multi-service environment.
- Add CI for lint/test/build.
- Add security checks.
- Create deployment pipeline.

## Key deliverables

- Dockerfile
- Compose/local environment
- CI
- Automated tests
- Build artifacts
- Deployment workflow

Exit condition: A clean checkout can be built and tested reproducibly and packaged for deployment.

## Phase 12 - Production Hardening

Goal: Validate the system against realistic failure and scale scenarios.

### Objectives

- Review indexes and connection pools.
- Tune rate limits and caching.
- Add timeout/retry policies.
- Perform load testing.
- Perform security testing.
- Review failure handling and observability.

## Key deliverables

- Performance review
- Load testing
- Security review
- Failure testing
- Capacity assessment

Exit condition: The system has a documented production-readiness assessment and known bottlenecks are addressed.

## Phase 13 - Future Commercialization

Goal: Extend only after an explicit product decision.

### Objectives

- Validate product-market needs first.
- Add file uploads/object storage.
- Add 2FA/OAuth/SSO.
- Add billing/subscriptions.
- Add audit/compliance features.
- Evolve membership model if needed.

### Key deliverables

- File storage
- 2FA
- OAuth
- SSO
- Billing
- Subscriptions
- Usage limits
- Audit logs
- Advanced analytics

Exit condition: Commercial features are introduced only after a product decision justifies them.

## Roadmap Gate Rules

- Do not start a phase until the previous phase's exit condition is met.
- Do not introduce infrastructure without a concrete product or operational use case.
- Security and tenant isolation are never deferred.
- Commercial features remain outside the learning roadmap until explicitly re-prioritized.

## Recommended Order

Foundation -> Authentication -> Organizations -> RBAC -> Projects/Tasks/Comments -> Query infrastructure -> Redis -> Kafka/background jobs -> Notifications -> Observability -> Testing -> Docker/CI/CD -> Production hardening -> Commercialization.